

UNIVERSIDAD NACIONAL GUILLERMO BROWN

Inteligencia Artificial — 1° Cuatrimestre 2026

TRABAJO PRÁCTICO FINAL

Desarrollo Progresivo de una Arquitectura de Red Neuronal

Docente: Pablo Moreira

Modalidad: Individual | Entrega: Notebook Jupyter comentado

Inicio: semana del 01/04/2026 | Entrega final: 24/06/2026

1. Introducción y objetivo general

Elegir un dataset y construir, paso a paso, un sistema de Machine Learning que evolucione desde un modelo baseline hasta una arquitectura generativa (VAE o GAN), aplicando los conceptos teóricos y prácticos de cada unidad.

La entrega es un único Notebook Jupyter (.ipynb) que crece semana a semana. Cada etapa tiene su propia sección claramente delimitada con celdas Markdown explicativas.

El objetivo no es que el modelo funcione, sino que puedas justificar cada decisión de diseño, interpretar los resultados y aplicar las técnicas vistas en clase para mejorar el sistema.

2. Elección de datasets

Ingresar a la planilla compartida y colocar el nombre en la columna de Alumno

[Planilla](#)

<https://docs.google.com/spreadsheets/d/14-2w1tDjpXddl01tQsrSoBdZFeXUycyhEIVZF0m5PhY/edit?usp=sharing>

3. Detalle de cada etapa

Etapa 1 — Exploración y modelo baseline

El objetivo es entender el dataset y establecer un punto de partida medible.

3.1.1 Carga y análisis exploratorio del dataset (EDA)

- Cargar el dataset usando la fuente indicada en la tabla de asignación.
- Describir las variables: tipos, rangos, distribuciones, valores nulos.
- Visualizar al menos 3 gráficos relevantes (histogramas, distribución de clases, ejemplos de imágenes si corresponde).
- Identificar posibles problemas: desbalance de clases, outliers, data mismatch potencial.

3.1.2 Partición de datos

- Dividir el dataset en tres conjuntos: Entrenamiento (70%), Desarrollo/Dev (15%) y Prueba/Test (15%).
- Justificar la estrategia de partición elegida (stratified, random, temporal, etc.).
- Verificar que la distribución de clases sea proporcional en los tres conjuntos.

3.1.3 Preprocesamiento y modelo baseline

- Aplicar el preprocesamiento mínimo necesario (normalización, encoding, reshape según el tipo de dato).
- Implementar un modelo baseline simple: puede ser regresión logística, árbol de decisión o una red densa de 1-2 capas.

- Definir y calcular una métrica de evaluación de número único apropiada (accuracy, F1-macro, AUC, etc.). Justificar la elección.
- Reportar el resultado del baseline en el conjunto Dev. Este valor será la referencia para todas las etapas siguientes.

Etapla 2 — Red neuronal multicapa y análisis de errores

En esta etapa se reemplaza el baseline por una red neuronal densa (MLP) y se aplica un ciclo sistemático de mejora.

3.2.1 Análisis de errores manual

- Tomar una muestra de al menos 50 ejemplos mal clasificados por el baseline en el conjunto Dev.
- Categorizar los errores: ¿son errores de sesgo (underfitting), de varianza (overfitting) o de data mismatch?
- Documentar hallazgos con ejemplos concretos y discutir qué cambios podrían corregir cada tipo de error.

3.2.2 Arquitectura MLP

- Diseñar e implementar una red neuronal densa con al menos 3 capas ocultas.
- Documentar la arquitectura: número de capas, neuronas por capa, función de activación, función de pérdida, optimizador.
- Graficar las curvas de aprendizaje (loss y métrica) para entrenamiento y validación.

3.2.3 Técnicas de regularización

- Aplicar al menos dos de las siguientes técnicas: Dropout, L2 regularization, Batch Normalization, Early Stopping, Data Augmentation (si aplica).
- Comparar el rendimiento con y sin regularización. Mostrar curvas de aprendizaje en ambos casos.
- Reportar la mejora obtenida sobre la métrica de número único respecto al baseline de la Etapa 1.

Etapla 3 — Arquitectura específica del dominio

Esta etapa varía según el tipo de dataset asignado. Ver la especificación correspondiente a continuación.

3.3.A Para datasets de imágenes

- Implementar una CNN con al menos 3 bloques convolucionales (Conv2D + ReLU + MaxPooling).
- Comparar el enfoque End-to-End (CNN directa) versus un pipeline secuencial (extractor de features + clasificador).
- Aplicar Data Augmentation (flip, rotación, zoom) y analizar su impacto en las curvas de aprendizaje.
- Realizar atribución de errores: ¿qué porcentaje de los errores del modelo proviene de la etapa convolucional vs. la etapa de clasificación?

- Reportar la mejora sobre la métrica de número único respecto a la Etapa 2.

3.3.B Para datasets de texto

- Implementar embeddings de palabras (se pueden usar embeddings pre-entrenados como GloVe o Word2Vec).
- Construir una red densa sobre los embeddings para clasificación o modelado de lenguaje simple.
- Comparar el uso de embeddings pre-entrenados vs. embeddings entrenados desde cero.
- Analizar qué tipos de ejemplos siguen siendo difíciles para el modelo. Documentar con al menos 10 ejemplos concretos.
- Reportar la mejora sobre la métrica de número único respecto a la Etapa 2.

Etapa 4 — Cierre, comparación y análisis ético (inicio: 27/05)

La etapa final integra todos los modelos desarrollados y añade una reflexión crítica sobre el trabajo realizado.

3.4.1 Evaluación final en Test

- Evaluar el modelo desarrollado en el conjunto Test (por primera y única vez en todo el trabajo).
- Comparar el resultado en Test con el resultado en Dev. Discutir si hay evidencia de overfitting al conjunto Dev.
- Reflexionar sobre qué mejoras adicionales se podrían aplicar con más tiempo o recursos.

3.4.2 Análisis ético y de sesgos

- Analizar si el dataset presenta sesgos que podrían afectar al modelo (sesgos de género, etnia, clase social, idioma, etc.).
- Evaluar el desempeño del modelo diferenciado por subgrupos relevantes del dataset. ¿El modelo funciona igual de bien para todos los grupos?
- Reflexionar sobre las implicancias de usar este modelo en un sistema real: ¿qué riesgos podría generar? ¿Qué medidas de mitigación propondrías?

3.4.3 Conclusiones

- Resumir los aprendizajes más importantes del trabajo.
- Indicar qué fue lo más desafiante y cómo se resolvió.
- Describir las limitaciones del trabajo y qué se haría diferente con más tiempo.

4. Formato y modalidad de entrega

4.1 Nombre del archivo

El notebook debe nombrarse con el siguiente formato:

| `TP_IA2026_APELLIDO_NombreDataset.ipynb`

Ejemplo: TP_IA2026_MOREIRA_CIFAR10.ipynb

4.2 Canal de entrega

La entrega se realiza a través del campus virtual de la Universidad.

Se habilitará la posibilidad de realizar entregas parciales para supervisión y consultas.

La entrega final de la Etapa 5 debe incluir el notebook completo con todas las etapas integradas.

5. Recomendaciones y recursos

5.1 Stack tecnológico recomendado

- Python 3.10+
- PyTorch 2.x o TensorFlow/Keras 2.x (se puede elegir, pero debe ser consistente en todo el trabajo).
- scikit-learn para baseline y métricas.
- matplotlib / seaborn para visualizaciones.

5.2 Recursos de referencia

- Andrew Ng — Machine Learning Yearning (PDF gratuito): referencia principal para bias/varianza y partición de datos.
- Deep Learning Book (Goodfellow et al.) — Capítulos 6, 9, 14, 20: arquitecturas MLP, CNN, VAE, GAN.
- PyTorch Tutorials (pytorch.org/tutorials): guías oficiales por tipo de tarea.
- Papers With Code (paperswithcode.com): resultados de estado del arte por dataset, útil para comparar el propio modelo.

5.3 Consejos para el éxito

- Empezar cuanto antes. La Etapa 1 puede completarse en una tarde; postergarla complica todo lo que sigue.
- Committear el notebook en GitHub o Google Drive con frecuencia para no perder avances.
- Documentar los experimentos fallidos: son tan informativos como los exitosos y suman en la evaluación.
- Ante dudas sobre el dataset asignado o la arquitectura a usar, consultar al docente a través de los medios habilitados (foros del Campus o mail).